

Recursion

$$\text{fact}(0) = 1 \quad \rightarrow \text{base case}$$

$$\text{fact}(n) = n * \text{fact}(n-1) \quad | \quad (n > 0)$$

$$\text{Power}(x, n) = \begin{cases} 1 & \text{if } n = 1 \\ x * \text{Power}(x, n-1) & \text{if } n > 0 \end{cases}$$

Page ①

```
#include <stdio.h>
```

```
long int fact(int n)
```

```
{  
    if (n == 0)  
        return 1;
```

```
    else  
        return n * fact(n-1);
```

```
}
```

```
int main()
```

```
{  
    int num;
```

```
    printf("Enter Number");
```

```
    scanf("%d", &num);
```

```
    printf("factorial of %d is %d", num, fact(num));
```

```
}
```

fact(5)
↓
return 5 * fact(4)

↓
return 4 * fact(3)

↓
return 3 * fact(2)

↓
return 2 * fact(1)

↓
return 1 * fact(0)

↓
return 1

⇒ Finding Factorial by Recursion.

fact(5)



return 5 * fact(4)



return 4 * fact(3)



return 3 * fact(2)



return 2 * fact(1)



return 1 * fact(0)

return 1

fact(5) = 120



return 5 * 24



return 4 * 6



return 3 * 2



return 2 * 1

return 1 * 11

$$\text{fact}(6) = 6 * \text{fact}(5)$$

in general form

$$\boxed{\text{fact}(n) = n * \text{fact}(n-1)}$$

(3)

```
int mult(int a, int b)
```

```
{ if ((a==0) || (b==0))
```

```
    return 0;
```

```
    if (b==1)
```

```
        return a;
```

```
    return a + mult(a, b-1); // b times sum of a
}
```

⇒ Recursive function
to multiply
two numbers.

i) $\text{mult}(0, 5)$

↓
return 0;

ii) $\text{mult}(5, 0)$

↓
return 0;

iii) $\text{mult}(5, 1)$

↓
return 5;

i) $\text{mult}(1, 4)$

↓
return 1 + $\text{mult}(1, 3)$

↓
return 1 + $\text{mult}(1, 2)$

↓
return 1 + $\text{mult}(1, 1)$

↓
return 1

$\text{mult}(1, 4) = 4$

↑
return 1 + 3

↑
return 1 + 2

↑
return 1 + 1

$$\boxed{\text{mult}(a, b) = a + \text{mult}(a, b-1)}$$

$$\text{mult}(3, 5) = 3 + \text{mult}(3, 4)$$

4

$\text{mult}(3, 4)$

$\text{mult}(3, 4) = 12$

return

$3 + \text{mult}(3, 3)$

— + —

return

$3 + 9$

return $3 + \text{mult}(3, 2)$

— — —

return

$3 + 6$

return $3 + \text{mult}(3, 1)$

— | — —

return

$3 + 3$

return 3

— — — — — 3

⑤ Input Two Numbers and Find
1st Number Raised to Power of 2nd Number by
Recursion.

```
#include <stdio.h>
```

```
int power (int x, int n)
```

```
{
```

```
    if (n <= 0)
```

```
        return 1;
```

```
    else if (n == 1)
```

```
        return x;
```

```
    else
```

```
        return x * power(x, n-1);
```

```
}
```

$\text{power}(5, 3) =$

$5 * \text{power}(5, 2)$

$$\boxed{\text{power}(x, n) = x * \text{power}(x, n-1)}$$

$\text{power}(3, 4)$

$\text{power}(3, 4) = 81$

$\swarrow$
 $\text{return } 3 * \text{power}(3, 3)$

$\nearrow$
 $\text{return } 3 * 27$

$\swarrow$
 $\text{return } 3 * \text{power}(3, 2)$

$\nearrow$
 $\text{return } 3 * 9$

$\swarrow$
 $\text{return } 3 * \text{power}(3, 1)$

$\nearrow$
 $\text{return } 3 * 3$

$\swarrow$
 $\text{return } 3;$

(6) important

Fibonacci series using Recursion.

1 1 2 3 5 8 13 21 ...

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

```
int fib(int n)
```

```
{ if (n==1 || n==2)
```

```
    return 1;
```

```
    else
```

```
        return (fib(n-1) + fib(n-2));
```

```
}
```

```
int main()
```

```
{
```

```
    int i, n;
```

```
    printf("How many terms");
```

```
    scanf("%d", &n);
```

```
    for (i=1; i<=n; i++)
```

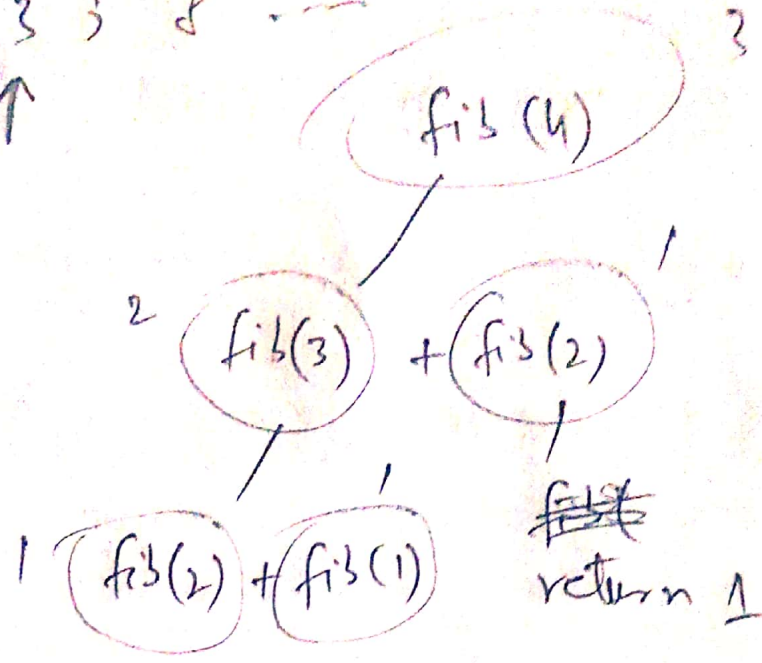
```
        printf("%d", fib(i));
```

```
}
```


1 1 2 3 5 8 ...

②

↑



1 1 2 3 5 8

↑

